

LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK
RELASI ANTAR OBJECT (CLASS DIAGRAM)

Disusun Oleh:

Muhammad Fharel

2511531010

Kelas Praktikum B

Tanggal Praktikum: 14 September 2026

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

A. Langkah Kerja Praktikum

Praktikum Modul 2 ini bertujuan untuk menerapkan konsep relasi antar objek dalam bentuk Asosiasi (One-to-Many). Pada modul sebelumnya, sistem hanya mampu mengubah saldo tanpa menyimpan rekam jejak aktivitas transaksi. Pada praktikum ini, setiap aktivitas penyetoran (Kredit) maupun penarikan (Debit) akan membentuk sebuah objek transaksi tersendiri dan disimpan di dalam koleksi objek milik rekening terkait.

1. Membuat Class Model Transaksi (Transaksi.java)

Class Transaksi bertindak sebagai model untuk merepresentasikan detail satu kejadian transaksi keuangan.

```
package pekan2_2511531010;  
  
public class Transaksi {  
    String idTransaksi;  
    String jenis;  
    double nominal;  
  
    // Constructor  
    public Transaksi(String id, String jenis, double nominal) {  
        this.idTransaksi = id;  
        this.jenis = jenis;  
        this.nominal = nominal;  
    }  
  
    public void cetakDetail() {  
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +  
" | Nominal: Rp" + nominal);  
    }  
}
```

Penjelasan Source Code Class Transaksi.java:

- State/Atribut (idTransaksi, jenis, nominal)

Merupakan variabel anggota yang melekat pada setiap objek transaksi secara mandiri. Atribut idTransaksi bertipe String untuk menyimpan kode unik entitas transaksi (misalnya kombinasi nomor alur dan waktu transaksi). Atribut jenis bertipe String digunakan untuk mencatat kategori transaksi, yaitu "Kredit" saat terjadi setoran tunai dan "Debit" saat penarikan tunai. Atribut nominal bertipe double untuk menampung besaran nilai dana riil yang diproses dalam transaksi tersebut.

- Constructor (public Transaksi(...))

Merupakan blok kode khusus yang dieksekusi secara otomatis saat objek transaksi dibuat menggunakan kata kunci new. Constructor menerima parameter id, jenis, dan nominal, lalu memetakan nilai masukan tersebut ke variabel state internal class menggunakan kata kunci this (this.idTransaksi = id;, dst.) untuk membedakan antara nama variabel atribut class dengan nama parameter lokal.

- Method cetakDetail()

Bertindak sebagai method pembaca (getter/display) yang bertugas mencetak ringkasan identitas dan status satu baris riwayat transaksi pengguna (meliputi ID, Jenis, dan Nominal) ke konsol CLI secara rapi tanpa melakukan perubahan nilai data di dalamnya.

2. Membuat Relasi pada Rekening.java

Buka kembali file Rekening.java dari praktikum sebelumnya. Selanjutnya, kita akan menambahkan atribut baru untuk mengimplementasikan relasi has-a, di mana satu rekening dapat menampung banyak transaksi.

```
package pekan2_2511531010;

import java.util.ArrayList;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Wajib menginisialisasi ArrayList di dalam constructor agar
        // tidak NillPointerException
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }
}
```

Penjelasan Source Code Class Rekening.java:

- Atribut Relasi Asosiasi (ArrayList<Transaksi> riwayatTransaksi)
Merupakan pemodelan relasi has-a (asosiasi one-to-many) antara class Rekening dan class Transaksi. Penggunaan struktur data dinamis ArrayList bertipe generik <Transaksi> memungkinkan satu objek rekening menyimpan banyak objek riwayat transaksi sekaligus secara dinamis tanpa batasan ukuran array yang kaku.
- Inisialisasi riwayatTransaksi di Constructor
Eksekusi instruksi this.riwayatTransaksi = new ArrayList<>(); di dalam constructor Rekening dilakukan untuk mengalokasikan ruang memori awal bagi koleksi riwayat saat objek rekening baru dibuat. Inisialisasi ini wajib dilakukan di dalam constructor untuk mencegah terjadinya error runtime NullPointerException saat method penyimpanan riwayat dipanggil di kemudian hari.

3. Mencatat Riwayat Transaksi

```
public void setorTunai(double nominal) {  
    if (nominal > 0) {  
        saldo += nominal;  
        // Merekam riwayat (Pembuatan objek Transaksi di dalam  
method)  
        String idTrx = "TRX-S-" + System.currentTimeMillis();  
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit",  
nominal);  
        riwayatTransaksi.add(trxBaru);  
  
        System.out.println("Setor tunai Rp" + nominal + "  
berhasil. Saldo saat ini: Rp" + saldo);  
    } else {  
        System.out.println("Gagal: Nominal setor harus lebih dari  
0!");  
    }  
}
```

Penjelasan Source Code Class Rekening.java:

- Perekaman Riwayat Transaksi Otomatis (setorTunai)
Pengubahan pada method setorTunai(double nominal) menambahkan logika pembentukan ID unik (TRX-S- dikombinasikan dengan waktu sistem System.currentTimeMillis()) dan instansiasi objek baru Transaksi

berjenis "Kredit". Objek transaksi yang terbentuk selanjutnya dimasukkan ke dalam atribut koleksi riwayatTransaksi menggunakan method .add() apabila proses penambahan saldo bernilai valid. Jika nominal kurang dari 0 maka transaksi dan saldo gagal ditambahkan.

B. Latihan Praktikum

Soal:

Tugas 1 : Integrasi Rekam Jejak Penarikan (30%)

- Buka kembali *method* tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

Tugas 2: Fitur Cetak Mutasi Rekening (70%)

- Buat sebuah *method* baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil *method* cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".
- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

Kode Program:

Tugas 1 : Integrasi Rekam Jejak Penarikan

Pada class Rekening.java

```
36 public void tarikTunai(double nominal) {
37     if (nominal < 10000) {
38         System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");
39     } else if (nominal > saldo) {
40         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
41     } else {
42         saldo -= nominal;
43
44         String idTrx = "TRX-T-" + System.currentTimeMillis();
45         Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
46         riwayatTransaksi.add(trxBaru);
47         System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
48     }
49 }
```

Penjelasan kode tugas 1:

- Validasi Nominal Minimum (if (nominal < 10000))
Menjamin batas minimal penarikan Rp10.000,0. Jika kurang, transaksi ditolak dengan pesan "Transaksi Gagal : Minimal nominal penarikan 10.000" tanpa mengubah saldo.
- Validasi Saldo (else if (nominal > saldo))

Membandingkan nominal dengan saldo. Jika penarikan melebihi saldo, transaksi dibatalkan dengan pesan "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[saldo]" untuk mencegah saldo minus.

- Eksekusi Penarikan & Perekaman Riwayat (else)

Jika lolos validasi, saldo dipotong (saldo -= nominal). Sistem membangkitkan ID unik ber-prefiks "TRX-T-" via System.currentTimeMillis(), menginstansiasi objek Transaksi berjenis "Debit", lalu menyimpannya ke riwayatTransaksi via .add().

Tugas 2: Fitur Cetak Mutasi Rekening

Pada class Rekening.java

```
public void cetakMutasi() {
    System.out.println("== RIWAYAT MUTASI REKENING ==");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi t : riwayatTransaksi) {
            t.cetakDetail();
        }
    }
    System.out.println("=====");
}
```

Penjelasan kode tugas 2:

- Pemeriksaan Riwayat Kosong (if (riwayatTransaksi.isEmpty()))

Mengecek keberadaan data transaksi. Jika belum ada transaksi, sistem mencetak pesan "Belum ada transaksi pada rekening ini".

- Penelusuran & Pencetakan Riwayat (else)

Jika riwayat terisi, blok else menjalankan perulangan for-each (for (Transaksi t : riwayatTransaksi)) untuk melintasi seluruh objek Transaksi di dalam ArrayList.

- Pemanggilan cetakDetail()

Di setiap iterasi, method t.cetakDetail() dipanggil untuk menampilkan rincian transaksi (ID, Jenis, dan Nominal) baris demi baris ke konsol.

Kode Program lengkap Class Rekening:

```
package pekan2_2511531010;

import java.util.ArrayList;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor,String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Wajib menginisialisasi ArrayList di dalam constructor agar
        // tidak NillPointerException
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek Transaksi di dalam
            // method)
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
            nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari
            0!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal : Minimal nomonal
            penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak
            mencukupi. Saldo Anda: Rp" + saldo);
        } else {
            saldo -= nominal;

            String idTrx = "TRX-T-" + System.currentTimeMillis();
```

```

        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
    }
}

public void cetakMutasi() {
    System.out.println("=== RIWAYAT MUTASI REKENING ===");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi t : riwayatTransaksi) {
            t.cetakDetail();
        }
    }
    System.out.println("=====");
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}
}

```

Contoh output:

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123456789
Masukkan Nama Pemilik: Jennie
Masukkan Saldo Awal: 500000
Rekening atas nama Jennie berhasil dibuat dengan saldo Rp500000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 5000
Transaksi Gagal : Minimal nominal penarikan 10.000

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 700000
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp500000.0

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Cetak Mutasi (Riwayat)
6. Keluar
Pilih menu: 1
Masukkan nominal tarik: 150000
Tarik tunai Rp150000.0 berhasil. Saldo saat ini: Rp350000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Cetak Mutasi (Riwayat)
6. Keluar
Pilih menu: 2
Masukkan nominal setor: 200000
Setor tunai Rp200000.0 berhasil. Saldo saat ini: Rp550000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Cetak Mutasi (Riwayat)
6. Keluar
Pilih menu: 6
===== RIMAYAT MUTASI REKENING =====
ID: TRX-T-1789916980660 | Jenis: Debit | Nominal: Rp150000.0
ID: TRX-S-1789917007395 | Jenis: Kredit | Nominal: Rp200000.0
=====

```

- Pembuatan Rekening: Pengujian diawali dengan memilih menu 1 untuk membuka rekening atas nama Fharel dengan saldo awal Rp500.000,0.
- Uji Coba Nominal di Bawah Minimal: Pengguna memilih menu 3 dengan nominal penarikan 5000. Program menolak transaksi karena kurang dari batas minimal Rp10.000,0 dan menampilkan pesan "Transaksi Gagal : Minimal nominal penarikan 10.000".
- Uji Coba Saldo Tidak Mencukupi: Pengguna mencoba menarik nominal 700000. Program membatalkan transaksi karena melebihi saldo aktif saat ini dan mencetak peringatan "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp500000.0".
- Uji Coba Penarikan Valid: Pengguna memasukkan nominal 150000. Karena memenuhi kriteria, method tarikTunai() berhasil memotong saldo menjadi Rp350.000,0, sekaligus membentuk objek Transaksi baru ber-prefiks "TRX-T-" dengan jenis "Debit" yang tersimpan di dalam ArrayList.
- Uji Coba Transaksi Tambahan: Pengguna melakukan setor tunai sebesar Rp200.000,0 via menu 2 untuk menambah catatan riwayat berjenis "Kredit".

- Eksekusi Cetak Mutasi: Pengguna memilih menu 6 untuk memanggil method cetakMutasi(). Program melintasi seluruh elemen di riwayatTransaksi menggunakan perulangan for-each dan memanggil cetakDetail().
- Verifikasi Hasil Mutasi: Konsol menampilkan daftar seluruh aktivitas keuangan yang pernah dilakukan (penarikan Debit ber-prefiks TRX-T- dan penyetoran Kredit ber-prefiks TRX-S-) secara runtut beserta nominal masing-masing. Jika belum ada transaksi sama sekali, program akan mencetak pesan "Belum ada transaksi pada rekening ini".

C. Link Repository GitHub

https://github.com/MuhammadFharel/PrakPBO_B_2511531010